Module 4 — Exception Handling

> A guaranteed topic. “Checked vs unchecked”, “can `finally` override a return”, and “how do you design exceptions in a REST API” show up constantly.

Node.js bridge: JS has one untyped `Error` and `try/catch`. Java has a **typed hierarchy**, a compiler-enforced **checked** category, and `try-with-resources` for deterministic cleanup.

4.1 Exception Hierarchy, Checked vs Unchecked, Error

1. Why Interviewers Ask This

Tests whether you understand recoverability and API design, not just `try/catch` syntax.

2. Core Concept

```
                Throwable
           +--------+--------+
         Error              Exception
      (don't catch)     +-------+---------+
   OutOfMemoryError   (checked)        RuntimeException (unchecked)
   StackOverflowError  IOException       NullPointerException
                       SQLException      IllegalArgumentException
                       ClassNotFound     IllegalStateException
                                         ArithmeticException
                                         IndexOutOfBounds
```

- **Checked** (compile-time): extend `Exception` (not `RuntimeException`). Compiler forces `throws` or `catch`. For **recoverable** external conditions (IO, DB).
- **Unchecked** (runtime): extend `RuntimeException`. For **programming errors** (bad args, null, illegal state). Not enforced.
- **Error**: serious JVM problems (`OutOfMemoryError`, `StackOverflowError`). Don’t catch/recover.

3. Internal Working

`throw` creates the exception object (capturing the **stack trace** at construction — expensive) and unwinds the stack frame by frame looking for a matching `catch`. If none is found in the thread, the thread’s `UncaughtExceptionHandler` runs and the thread dies. Building the stack trace (`fillInStackTrace`) is the costly part — never use exceptions for control flow.

4. Memory Diagram

```
throw new X()  --> unwind stack:
  frame f3  (no catch) pop
  frame f2  (catch X?) yes -> handle
  frame f1
If unwinds past main() -> UncaughtExceptionHandler -> thread terminates
```

5. Real Production Example

In Spring, a `@RestControllerAdvice` with `@ExceptionHandler` maps exceptions to HTTP: `EntityNotFoundException → 404`, `MethodArgumentNotValidException → 400`, everything else → 500. Checked `IOException` from a file/HTTP call gets wrapped in a domain `RuntimeException` so service signatures stay clean.

6. Most Asked Questions

- Checked vs unchecked — definition and when to use each?

- Is `NullPointerException` checked or unchecked? *(unchecked)*
- Can you catch an `Error`? Should you? *(can, shouldn't)*
- Difference between `throw` and `throws`?
- What's the cost of throwing? *(stack trace capture.)*

7. Traps

- Catching `Exception` or `Throwable` broadly and swallowing it (empty catch).
- Using exceptions for normal control flow.
- Catching then rethrowing while losing the cause (always chain: `new X(msg, cause)`).
- Confusing `throw` (statement) with `throws` (declaration).

8. Best Answer

> "Checked exceptions extend `Exception` and are compiler-enforced for recoverable external failures like IO; unchecked extend `RuntimeException` for programming bugs. `Error` is for JVM-level failures I never catch. Throwing captures a stack trace, so it's expensive — I never use it for control flow, always chain the cause, and in Spring I centralize mapping in a `@RestControllerAdvice`."

9. Coding Example

```
public class OrderService {
    Order place(String id) {
        try {
            return gateway.charge(id);          // may throw IOException (checked)
        } catch (IOException e) {
            // wrap checked -> domain unchecked, preserve cause
            throw new PaymentFailedException("charge failed for " + id, e);
        }
    }
}
class PaymentFailedException extends RuntimeException {
    PaymentFailedException(String m, Throwable cause){ super(m, cause); }
}
```

10. Follow-ups

- When would you make a custom exception checked vs unchecked? *(unchecked by default in modern code/Spring)*
- How does exception chaining help debugging? *(preserves root cause + full trace)*

11 & 12. Summary + Cheat

Checked=recoverable/enforced; Unchecked=bugs; Error=JVM. Chain causes; don't swallow.

4.2 try-catch-finally · try-with-resources · throw/throws

finally semantics (favorite trap)

- `finally` **always** runs — even on `return`/`throw` in try/catch. Only skipped by `System.exit()` or JVM crash.
- A `return` (or `throw`) in `finally` **overrides** any prior return/exception — an anti-pattern that swallows exceptions. Never `return` from `finally`.

```
int f() {
    try { return 1; }
    finally { return 2; }   // returns 2, swallows the 1 -> DON'T do this
}
```

try-with-resources (the modern way)

Any `AutoCloseable`/`Closeable` declared in the `try(...)` header is closed automatically in reverse order, even on exception — replacing verbose `finally { conn.close(); }`.

```
try (var conn = dataSource.getConnection();
     var ps   = conn.prepareStatement(SQL)) {
    ps.execute();
}   // ps then conn closed automatically, exceptions suppressed & attached
```

- Exceptions thrown by `close()` are **suppressed** and attached to the primary (`getSuppressed()`).
- Cleaner and leak-proof vs manual `finally`.

throw vs throws · multi-catch

- `throw` — actually throws an instance: `throw new IllegalArgumentException("x")`.
- `throws` — declares a method *may* throw: `void read() throws IOException`.
- **Multi-catch**: `catch (IOException | SQLException e)` — one block, `e` is effectively final; can't be a supertype/subtype pair.

4.3 Custom Exceptions

Best practice

```
public class InsufficientFundsException extends RuntimeException {   // unchecked by default
    private final String accountId;
    public InsufficientFundsException(String accountId, Throwable cause) {
        super("Insufficient funds for account " + accountId, cause);  // message + cause
        this.accountId = accountId;
    }
    public String getAccountId() { return accountId; }
}
```

- Extend `RuntimeException` unless callers can meaningfully recover (then `Exception`).
- Include context (ids, values) and preserve the cause.
- Give it a clear name ending in `Exception`.
- In Spring, map with `@ResponseStatus` or `@ExceptionHandler`.

Spring global handler (production pattern)

```
@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(EntityNotFoundException.class)
    ResponseEntity<ApiError> notFound(EntityNotFoundException e){
        return ResponseEntity.status(404).body(new ApiError("NOT_FOUND", e.getMessage()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ApiError> badRequest(MethodArgumentNotValidException e){
        return ResponseEntity.badRequest().body(new ApiError("VALIDATION", e.getMessage()));
    }
}
```

Module 4 — Top 25 Interview Questions (senior answers)

1. **Exception hierarchy?** Throwable → Error / Exception → RuntimeException.
2. **Checked vs unchecked?** Compiler-enforced recoverable vs programming bugs.
3. **Is Error catchable?** Technically yes; never recover from it.

4. **throw vs throws?** Throw an instance vs declare possibility.
5. **Does finally always run?** Yes, except `System.exit`/JVM crash.
6. **Can finally override return?** Yes — anti-pattern; avoid returning in finally.
7. **try-with-resources?** Auto-closes AutoCloseable in reverse order; suppresses close exceptions.
8. **Suppressed exceptions?** From `close()`, attached via `getSuppressed()`.
9. **Multi-catch rules?** `A | B`, effectively final, no sub/supertype overlap.
10. **NPE checked/unchecked?** Unchecked (RuntimeException).
11. **Custom exception: checked or unchecked?** Prefer unchecked in modern/Spring code.
12. **Exception chaining?** Pass cause to constructor to keep root cause.
13. **Cost of throwing?** Stack trace capture; don't use for control flow.
14. **Catch order?** Subclass before superclass or compile error.
15. **finally vs finalize vs final?** Cleanup block vs deprecated GC hook vs modifier.
16. **Rethrow best practice?** Wrap with cause, add context, don't swallow.
17. **Checked exception in a stream lambda?** Not allowed — wrap in unchecked or use helper.
18. **What is a stack trace?** Call chain captured at construction.
19. **When wrap checked → unchecked?** To keep service signatures clean / cross layers.
20. **Handling exceptions in Spring REST?** `@RestControllerAdvice` + `@ExceptionHandler`.
21. **`@ResponseStatus`?** Maps an exception to an HTTP status.
22. **Global vs local handling?** Centralize cross-cutting; local for specific recovery.
23. **Difference: `printStackTrace` vs logging?** Use a logger; never `printStackTrace` in prod.
24. **Can constructors throw?** Yes.
25. **What happens to uncaught exception in a thread?** UncaughtExceptionHandler; thread dies.

Module 4 — Top Coding Questions

- Implement a domain exception hierarchy + Spring `@RestControllerAdvice`.
- Convert legacy `finally { close(); }` to try-with-resources.
- Write a method that wraps a checked exception in a lambda (`Function` wrapper).
- Predict output of tricky try/finally/return snippets.

Module 4 — Common Follow-ups

- "Would you make this exception checked? Why not?"
- "What happens to a `close()` exception in try-with-resources?"
- "How do exceptions behave inside a stream pipeline?"

Module 4 — One-Page Cheat Sheet

```
Throwable -> Error (don't catch) | Exception -> checked (IOException) / RuntimeException (unchecked)
Checked = recoverable, compiler-enforced (throws/catch). Unchecked = bugs.
throw=instance ; throws=declaration ; multi-catch (A|B)
finally always runs (except System.exit); never return in finally
try-with-resources: AutoCloseable, reverse close, suppressed exceptions
Custom: extend RuntimeException, add context, chain cause
Spring: @RestControllerAdvice + @ExceptionHandler / @ResponseStatus
Never: swallow, use for control flow, printStackTrace in prod
```

Module 4 — Mock Interview (answer, then continue)

1. “Design the exception strategy for a REST API — which layers throw, which translate to HTTP?”
2. “What does a `try { return 1; } finally { return 2; }` return, and why is it dangerous?”
3. “A checked `IOException` bubbles up from a stream lambda — how do you handle it?”
4. “Walk me through what try-with-resources compiles to and what suppressed exceptions are.”
5. “When do you create a checked vs unchecked custom exception?”

Continue to Module 5 when ready.